Design and Evaluation of an Accessible IDE Plugin for Supporting Neuro-divergent Software Developers

Clo Elis

220402088

May 2026

Computer Science

Supervisor: Dr. Vlad Gonzalez

Abstract (NEED TO BE REWRITTEN) Modern Integrated Development Environments (IDEs) are designed for maximum functionality, yet their high feature density creates significant barriers for neurodivergent software developers. Individuals with Attention-Deficit/Hyperactivity Disorder (ADHD), Autism Spectrum Disorder (ASD), or dyslexia often face challenges related to executive dysfunction, including cognitive overload, difficulty initiating tasks, and sensitivity to interruptions in workflow. This research explores the extent to which an Artificial Intelligence (AI) integrated IDE plugin can mitigate these challenges.

The project involves the development of a custom VS Code extension featuring three core modules: a Minimalist Toggle to reduce interface-driven distractions, a Task Architect that uses Large Language Models (LLMs) to decompose complex prompts into manageable checklists, and a Visual Styler providing high-contrast, low-strain presets. The effectiveness of the plugin is evaluated through a comparative study using a baseline VS Code "Vanilla" versus the enhanced environment. Success is measured via the NASA Task Load Index (NASA-TLX) [3] to quantify mental demand and the System Usability Scale (SUS) [7] to assess overall tool effectiveness.

By streamlining the development environment and providing AI-assisted task structuring, this project aims to lower the "barrier to entry" for neurodivergent developers, allowing them to reallocate cognitive resources from navigating complex tools to solving core programming problems.

Declaration “I declare that this document represents my own work except where otherwise stated” - Clo Ellis

Acknowledgments

Contents

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Problem Statement . . . 1
1.3 Understanding Neurodiversity in Software Development . . . 2
1.4 Aims and Objectives . . . 2

2 Literature Review **5**
2.1 Cognitive Load Theory (CLT) . . . 5
2.2 Neurodiversity in Software Engineering . . . 6
2.3 The "Double-Edged Sword" of Hyperfocus . . . 8
2.4 Existing Tools . . . 8
2.5 Colour filters . . . 9
2.6 AI as a Cognitive Tool: Task Decomposition and Scaffolding . . . 10

3 Design and Planning **11**
3.1 User Requirements . . . 11
3.2 Module Design . . . 11
3.2.1 Minimalist Toggle . . . 11
3.2.2 Task Architect . . . 11
3.2.3 Visual Styler . . . 11
3.3 System Architecture . . . 12
3.4 Implementation . . . 13
3.5 Technology Stack . . . 13
3.6 UI Simplification and Minimalist Toggle . . . 14
3.7 AI Integration . . . 14
3.8 Challenges Overcome . . . 14
3.9 Evaluation and Methodology . . . 14
3.9.1 Experimental Design . . . 14
3.9.2 Qualitative Frameworks . . . 15

4 Results and Discussion **16**
4.1 Analysis of NASA-TLX Scores . . . 16
4.2 Task Initiation Findings . . . 16
4.3 Discussion . . . 16
4.4 Limitations . . . 16

References **17**

1 Introduction

1.1 Motivation

This project was initially motivated by my lived experience as a neurodivergent developer and by personal challenges in using industrial software development tools. Once I started talking to other developers and researching the area, I became even more motivated to find a solution to these issues.

Existing IDEs can lead to increased cognitive overload and difficulty in maintaining focus. By streamlining interfaces and providing AI-driven task decomposition, this project aims to reduce the strain on neurodivergent individuals in both professional and educational settings, as developers often report challenges with cognitive overload, hyperfocus, and breaking down more complex problems when programming

1.2 Problem Statement

IDEs are a code tool for modern software development, but there are still elements of the interface that remain a significant barrier for neurodivergent developers, specifically those with ADHD, ASD, and related executive functioning challenges. Current IDEs are designed with "maximalist" in mind, presenting a constant stream of visual stimuli, complex nested menus, and dense technical jargon.

For developers who experience sensory overwhelm or executive dysfunction, modern IDEs can create an immense Extraneous Cognitive Load [15]. Research by Bramham et al. [2] found that groups with ADHD often battle with task paralysis during the initiation phase, while those with ASD may struggle more with the initial strategic planning and decomposition of a project.

Constant notifications from compiler errors and recommendations can trigger hyperfocus on peripheral details rather than the core logic. There is currently a growing resilience on AI tools, with developers using them to write their code. Even if they are using the models as a form of support, they often provide answers rather than acting as a "cognitive prosthetic" that doesn't just help with code, but helps with the process of being a developer by filtering noise and decomposing complexity.

Here is a more precise breakdown of the challenges and potential strengths of neurodiverse people regarding their conditions. Obvously the challenges faced by individuals can be deeply varied and nuanced, but this should provide enough of an overview for you to understand the key points and aims of the project.

1.3 Understanding Neurodiversity in Software Development

Before going into the specific technical barriers of modern IDEs, it is necessary to define neurodiversity and the cognitive frameworks that underpin this research. Originally coined by Judy Singer in the late 1990s, "neurodiversity" views neurological differences—such as ASD, ADHD, and Dyslexia, as natural variations in the human genome rather than purely medical deficits [14].

In the context of software engineering, these variations often manifest as unique cognitive profiles. While many neurodivergent developers possess high aptitude for logic and pattern recognition, they frequently encounter barriers related to Executive Functioning. This is an umbrella term for the mental processes that enable us to plan, focus attention, remember instructions, and juggle multiple tasks successfully. For a developer, executive dysfunction might manifest as task paralysis when faced with a blank text editor, or an inability to filter out the visual noise of a complex interface.

To provide a clearer lens for the objectives of this project, the below table outlines the specific strengths and environmental challenges associated with the conditions most prevalent in the developer community.

Table 1: Neurodivergent Strengths and Environmental Challenges

Condition	**Potential Strengths**	**Common Environmental Challenges**
ASD	Deep focus, systematic thinking, high integrity, exceptional memory for details, strong pattern recognition.	Sensory overload in noisy or bright environments, misinterpretation of direct communication style, difficulty with unspoken social rules.
ADHD	Creativity, hyperfocus on passions, adaptability, high energy, ability to multitask or rapidly switch contexts.	Distractibility in chaotic settings, struggles with time management and organisation, impulsivity in structured environments.
Dyslexia	Strong visual-spatial reasoning, excellent big-picture thinking, creative problem-solving, strong verbal skills.	Difficulties with text-heavy tasks, timed reading assignments, and conventional written communication.

1.4 Aims and Objectives

Project Aim

The primary objective of this project is to design, implement, and evaluate an assistive VS Code plug-in that reduces cognitive overload and executive dysfunction for neurodivergent software developers. By introducing a "cognitive prosthetic" layer, the project seeks to determine if targeted User Interface (UI) simplification and AI-driven task decomposition can measurably improve the developer experience by minimising extraneous cognitive load.

Research Objectives

Objective 1: Interface Optimisation (The Minimalist Toggle)

The first aim will be to create a "Minimalist Toggle" using the VS Code Extension to allow users to instantly close all non-essential UI elements. The toggle hides the Activity Bar, Status Bar, Mini-map, and Breadcrumbs at the same time, reducing the need for extensive menu navigation. This addresses the "Extraneous Cognitive Load" identified by [15], reducing visual noise that leads to distraction and sensory overwhelm for autistic and ADHD developers.

Objective 2: AI-Driven Task Decomposition (The Task Architect)

I will develop a LLM integration that translates complex, ambiguous programming goals into structured, editable checklists. This element will require the implementation of a "Chain of Thought (CoT)" prompting strategy ([11]) via the OpenAI Application Programming Interface (API) that breaks down high-level requirements without providing the direct code solution. This supports "Task Initiation" and "Executive Functioning," helping users overcome the paralysis often associated with large, unstructured projects ([4]).

Objective 3: Simplification of Technical Feedback (The Code Explainer)

To build a module that intercepts complex compiler error messages and re-interprets them into plain-language, direct instructions. Limit output to a maximum of three bullet points using a "Direct Instruction" style [12]. Justification: This minimises the cognitive demand of interpreting dense technical jargon during high-stress moments (such as debugging), preventing hyperfocus "rabbit holes."

Objective 4: Visual Accessibility Customisation (The Visual Styler)

To implement a suite of visual presets focused on reducing eye strain and enhancing readability. Target: Develop at least two high-contrast and low-stimuli themes that adhere to WCAG 2.1 contrast guidelines [1]. Justification: Addresses the sensory sensitivities and "attractive colour" preferences reported by neurodivergent developers in recent surveys [12].

Objective 5: Empirical Evaluation of Cognitive Impact

To conduct a comparative study assessing the plugin's efficacy against a standard IDE

environment. Target: Collect and analyse quantitative data via the NASA-TLX and the SUS. Justification: This provides a standardised scientific metric to validate whether the tool successfully reduces mental demand and improves the workflow for the target demographic.

2 Literature Review

2.1 Cognitive Load Theory (CLT)

Cognitive Load Theory, primarily developed by John Sweller, suggests that since the human brain has limited working memory capacity, instructional design can aid in avoiding overload.

Sweller's Tripartite Framework of Load

Sweller [15] proposes a three-part load framework that identifies three different types of cognitive load that impact a learner's ability to process information:

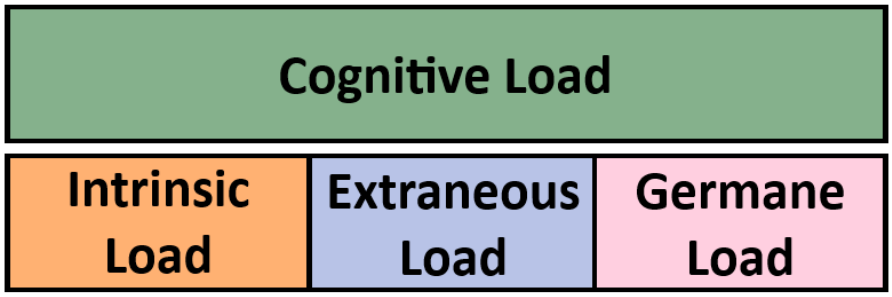


Figure 1: Sweller's Tripartite Model of Cognitive Load, illustrating the balance between Intrinsic, Extraneous, and Germane loads.

1. Intrinsic Load: This is the inherent difficulty of the task itself. It is defined by the number of elements must be processed simultaneously to understand a concept. For example, learning basic vocabulary has low intrinsic load, while mastering complex syntax has high intrinsic load.

2. Extraneous Load: This is seen as the unnecessary load that you would aim to minimise wherever possible. It is influenced by the way information is presented. Some examples that could exacerbate the load are poorly designed diagrams, distracting environments or redundant text. Poor design choices force the brain to waste resources on non-essential processing.

3. Germane Load: This is the load I am aiming to maximise. It refers to the mental effort that can be directed toward the more key task, which in this scenario would be the code programming task.

The key role of CLT is to manage intrinsic load, break down tasks, minimise extraneous load, facilitate a clear design, and maximise germane load, allowing the user to focus on their core task.

Integration with Rosenshin‘s Principles of Instruction

While Sweller provided the theoretical framework, Barack Rosenshine provided a “how-to” to be used in classrooms or digital environments [13].

Key intersections include:

- Small Steps (Principle 2): By presenting new material in small increments, the instructor limits the Intrinsic Load, ensuring the working memory is not overwhelmed at the outset.
- Scaffolding (Principle 7): Providing temporary supports reduces Extraneous Load, allowing the learner to focus their cognitive resources on the core task rather than the “noise” of the process.

To view these from more of a development lens, principle 2 supports breaking down the core task into smaller, more manageable steps. Further more these smaller task can be seen as “scaffolding” for the user when they are beginning a task without handing them the answer directly.

The Cognitive Theory of Multimedia Learning (CTML) While Sweller focuses on the load, Richard Mayer‘s CTML provides a granular look at how visual and auditory channels process information. In the context of a software developer, the IDE is a high-density multimedia environment.

The Split-Attention and Redundancy Principles Mayer (2021) identifies the Split-Attention Effect, which occurs when learners must divide their attention between multiple sources of information that are physically separated but cognitively related. For a neurodivergent developer, an error in the "Output" console that refers to a line of code 500 lines above creates a massive "spatial split." The mental effort required to integrate these two sources is entirely Extraneous Load.

Furthermore, the Redundancy Principle suggests that humans learn better from graphics and narration than from graphics, narration, and on-screen text combined. In modern IDEs,“hover-over” documentation often repeats exactly what is written in the type-hinting, adding "visual noise" that can trigger sensory overload in developers with ASD.

2.2 Neurodiversity in Software Engineering

To best design an inclusive plug-in, I needed to gain a deep understanding of the specific cognitive roadblocks faced by neurodivergent developers. Recent research by Verma et al. [16] and Gama et al. [4] highlights that developers with ADHD, ASD, and Dyslexia face significantly higher frequency of interruptions and difficulty obtaining knowledge.

The Cognitive Load Connection

For a neurodivergent developer, a standard IDE error message often carries a high Extraneous Load. What a neurotypical developer might perceive as a simple notification, a neurodivergent individual may experience as a wall of cryptic text, inconsistent highlighting, and ambiguous phrasing. This noise competes for limited working memory resources, leaving little room for the Intrinsic Load required actually to solve the logical problem.

executive dysfunction and Task Switching

executive dysfunction is a primary barrier cited by both [16] and [4]. This manifests itself as difficulty in “task initiation” (starting a new project) and “set-shifting” (switching between tasks).

The Industry Context: In a modern software environment, developers are rarely afforded the luxury of a single focus; they must balance code reviews, meetings, and multiple projects.

The Impact: For those with ADHD, the mental cost of “re-loading” the context of a project after an interruption is significantly higher. If the IDE does not provide clear "waypoints" or historical context, the developer may become cognitively paralysed, unable to determine where they left off.

Specific Challenges by Condition

Based on the synthesis of recent studies, the challenges can be categorised as follows:

ADHD: Frequent loss of“state” due to external interruptions and internal distraction. [16] found that these developers report higher stress related to “waiting for answers,” as delays in information retrieval often lead to a total loss of task momentum.

ASD: Challenges with the “Double Empathy” problem in code—where the high-level abstractions or “implied logic” used by neurotypical colleagues create a high cognitive overhead for autistic developers who prefer explicit, literal documentation.

Dyslexia: Significant hurdles during the “syntax-heavy” stages of programming. While these developers often excel at high-level architectural mapping, the Extraneous Load of spelling-sensitive compiler errors and dense documentation can be a major deterrent to productivity.

Beyond Medical

To understand why existing tools fail, we must examine the shift from the Medical Model to the Social Model of disability.

- The Medical Model: Traditionally, ADHD or Dyslexia in engineering has been viewed as a “deficit” to be corrected through individual coaching or medication.
- The Social Model: This perspective argues that the developer is “disabled” not by

their neurology, but by an IDE designed with a “neurotypical bias”. The problem is not the individual’s brain, but the environment that fails to accommodate it.

Executive Functioning (EF) Domains in Programming To deepen the analysis of executive dysfunction, we must break it into three core domains:

- Inhibitory Control: The ability to ignore distractions. For ADHD developers, the “pop-up” notifications of a linter can break a fragile state of focus.
- Working Memory: The “mental scratchpad.” Dyslexic developers often have a high capacity for 3D spatial reasoning but a limited verbal working memory, making syntax-heavy languages like C++ more taxing than “cleaner” languages like Python.
- Cognitive Flexibility: The ability to switch between thinking about the “big picture” architecture and the “low-level” syntax.

2.3 The "Double-Edged Sword" of Hyperfocus

While neurodivergent developers, particularly those with ASD and ADHD, are often stereotyped as having a “superpower” for deep work, this ability to hyperfocus is more accurately described as a dysregulation of attention. As noted by [4], hyperfocus allows for an exhaustive exploration of complex problems, yet this intensity often comes at the expense of self-care. Developers may neglect fundamental biological needs, such as nutrition, sleep, and hygiene, leading to rapid burnout. Furthermore, this state is notoriously difficult to achieve, [10] identifies that for ADHD programmers, “getting in the groove” is a fragile process where a single interruption can collapse hours of built-up mental context.

In a professional engineering context, this creates a significant conflict between depth and delivery. Hyperfocus can inadvertently trigger over-engineering, where a developer becomes "locked" into optimising a single component while neglecting broader project milestones or team collaborations. This inability to switch off from an interesting problem makes task-switching, a fundamental requirement of agile development, mentally exhausting and practically difficult. Consequently, what appears to be peak productivity from the outside is often an unsustainable trade-off that risks both the developer’s well-being and the project’s timeline.

2.4 Existing Tools

The design of IDEsoften follows a one-size-fits-all model, which inherently leads to a neurotypical bias, creating barriers for those who may require additional support. [9]

observes workplace tools are frequently built with assumptions that "may not align with the needs of neurodiverse employees." IDEs are often packed with tools that enable customisation; however, they are not designed with neurodiverse developers in mind, and complex nested menus can be overwhelming.

Despite the obstacles described, certain features provide essential support for developers. [9] notes that tools such as code completion and linters are vital to alleviate the difficulties associated with dyslexia, such as syntax memorisation and spelling.

However, the burden of making tools accessible often falls to the user. [8] found that developers often create “individual strategies and tool configurations to support their productivity." While these features are currently helping, the reliance on manual customisation implies a lack of "out-of-the-box" inclusivity.

This project aims to bridge the gap between the availability of generic tools and their functional utility. While current tools are excellent at detecting errors, they often fail at explaining them in an accessible manner. By integrating an Error Explainer that decomposes complex compiler messages into manageable tasks, therefore reducing the cognitive load required to debug. This shifts the IDE from a passive environment that requires expert configuration to an active partner in the development process.

2.5 Colour filters

Visual presentation can influence reading performance and comfort. [5] found that coloured overlays may reduce visual stress for some readers, although the benefits vary significantly between individuals. This suggests that software tools should support flexible visual customisation rather than relying on a single accessibility configuration.

In some cases, high-contrast models are essential because they provide the necessary visual scaffolding to combat task paralysis. For developers with ADHD, a low-contrast or "aesthetic" pastel theme can cause code elements to bleed together, increasing the cognitive effort required to distinguish between keywords, variables, and logic.

By utilising high-contrast ratios (such as 7:1 or higher, per WCAG 2.1 guidelines), the IDE creates a clear hierarchy. This high "signal" allows the developer to:

- Reduce Visual Search Time: Finding the start and end of a function block becomes instantaneous rather than a manual scan.
- Mitigate the "Blur" of Hyperfocus: As noted by [10], the intense mental strain of being "in the groove" can lead to visual fatigue; high contrast maintains legibility even as physiological focus wanes.
- Support Syntax Logic: In complex, nested code, high-contrast separation between

brackets and operators serves as a "visual prosthetic," offloading the work of pattern recognition from the brain to the interface.

However, the implementation must be nuanced. While high contrast is vital for clarity, "pure" high contrast can cause halation (a glowing effect around letters) for those with ASD or visual stress. Therefore, the most effective high-contrast models for neurodiversity are often "Soft High Contrast", utilising high-visibility colours against deep non-pure-black backgrounds.

(ADD IMAGES)

2.6 AI as a Cognitive Tool: Task Decomposition and Scaffolding

For developers with ADHD and ASD, the most significant barrier to productivity is often not the complexity of the code itself, but the executive function tax required to organise a project. Research by [10] indicates that ADHD professional programmers specifically struggle with "state recovery" and task initiation. LLMs offer a unique solution to this through cognitive scaffolding. This process handles the "low-value" mental energy drains, such as formatting and scheduling, leaving more room for "high-value" deep thinking ([6]).

CoT as Task Decomposition is one of the most effective applications of LLMs for neurodivergent developers. Instead of asking a model to simply "write a function," the developer can use the model to decompose a broad, abstract goal into a granular, step-by-step plan. This process mitigates the "activation energy" barrier identified in ADHD populations. By prompting a model to "think step-by-step," the AI generates a logical roadmap for the developer to work through, as well as referring back to it if the task is interrupted.

As noted by Mew (2025), this "automated pattern" of scaffolding can increase productivity for developers with ADHD by up to 55 percent. Existing tools already demonstrate this by taking a single overwhelming command (e.g., "build an authentication system") and breaking it into manageable milestones. This effectively transforms AI from a simple code generator into a "digital ramp" that bypasses the freeze response common in task paralysis ([6]).

3 Design and Planning

3.1 User Requirements

The requirements for the plug-in were derived from a synthesis of Cognitive Load Theory and specific neurodivergent accessibility guidelines, notably the "Direct Instruction" and "Visual Comfort" frameworks identified by Rocha [12].

3.2 Module Design

3.2.1 Minimalist Toggle

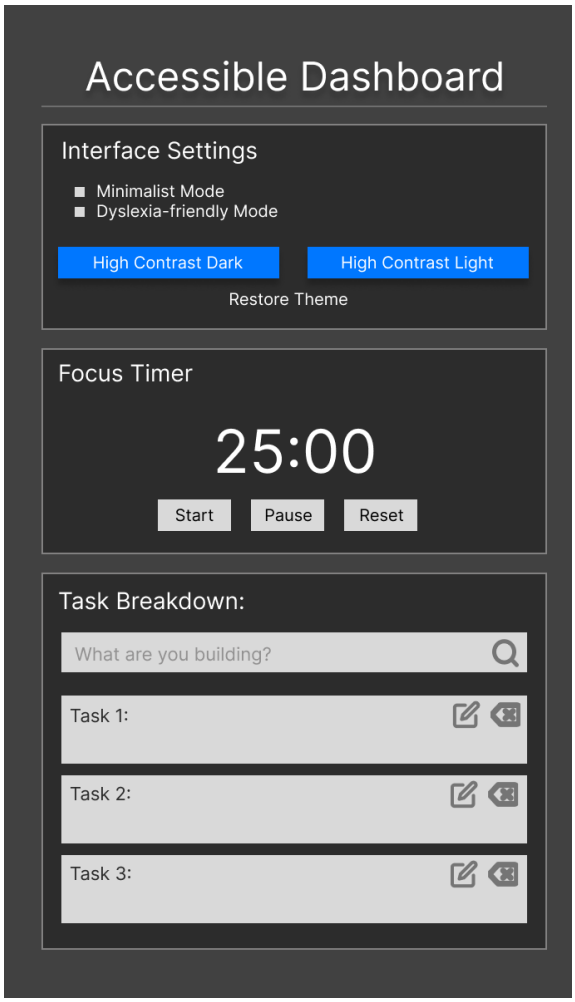
Logic for UI state management.

3.2.2 Task Architect

Prompt engineering strategy (HITL approach).

3.2.3 Visual Styler

WCAG 2.1 compliance mapping.



3.3 System Architecture

The system is divided into three primary layers: the Presentation Layer (Webview), the Orchestration Layer (Extension Host), and the External Services Layer (VS Code API).

1. The Presentation Layer: WebView UI

The user interface is implemented as a `WebviewPanel`, which renders a custom HTML/JavaScript dashboard. This allows for a highly flexible UI. This allows for the extension to be customised to best fit the user's needs, predetermined through prior research. This also allows for easier changes as more research is collected in the future

- State Communication: Because WebViews are isolated from the main editor, communication is handled via asynchronous message passing. The UI dispatches JSON-formatted messages using `vscode.postMessage()`, which are then intercepted by the extension's backend.
- Asynchronous Feedback: The UI remains responsive by using event listeners that wait for `updateTime`, `taskResult`, or `errorResult` commands sent back from the extension host.

2. The Orchestration Layer: Extension Host

The core logic resides in the activate function, which serves as the central hub while the extension is active.

- Command Registration: The system registers specific URI commands (e.g., accessible-toggle.showUI) that map user intents to internal JavaScript functions.
- Configuration Management: A key architectural feature is the interaction with vs-code.workspace.getConfiguration(). The extension reads and writes global settings to modify the editor's environment (font size, themes, and layout) in real-time, effectively acting as a wrapper for the VS Code "Settings" engine.
- Context Preservation: To ensure the system is non-destructive, the architecture utilises `context.globalState`. This allows the extension to "remember" the user's original theme and font settings before applying accessibility modifications, enabling a seamless restoration process. This also acts as a form of error prevention in case any settings are applied by accident.

3. The Functional Providers

The architecture employs specialised logic blocks to handle distinct accessibility needs:

Table 2: System Component Interaction and API Dependencies

Component	**Interaction Strategy**	**VS Code API Dependency**
Interface Controller	Imperative Commands	`commands.executeCommand`
Theme/Font Engine	Configuration Updates	`workspace.getConfiguration`
Task Architect	AI Stream Processing	`vscode.lm` (Language Model)
Code Helper	Diagnostic Polling	`languages.getDiagnostics`

4. Integration with Language Models

The "Task Architect" and "Code Helper" features utilise the VS Code Language Model API. This architectural choice ensures security and efficiency, as the extension does not require individual API keys from the user. Instead, it requests access to the editor's resident models (e.g., GPT-4o). The system uses a CoT prompting strategy to process raw diagnostic data or user goals into structured, neurodiverse-friendly steps before pushing the data back to the Presentation Layer.

3.4 Implementation

3.5 Technology Stack

JavaScript, Node.js, and OpenAI API.

3.6 UI Simplification and Minimalist Toggle

The implementation of the "Minimalist Mode" focuses on reducing visual noise, which is a primary driver of cognitive overload. This is achieved through an imperative command pattern:

- **Workspace Manipulation:** Using `vscode.commands.executeCommand`, the plugin toggles the visibility of the Activity Bar, Side Bar, Status Bar, and Breadcrumbs.
- **Global Configuration:** To disable the Minimap, the system interacts directly with the global user settings via `config.update('editor.minimap.enabled', ...)`.

3.7 AI Integration

The AI-driven features are implemented using a Streaming Data Pattern.

- **Prompt Engineering:** The *Task Architect* uses a strictly defined system prompt to enforce a "Micro-step" structure (3–5 sub-tasks with two 5-minute micro-steps each). This prevents "Wall of Text" responses that can be overwhelming.
- **Code Explainer:** This module polls the `vscode.languages.getDiagnostics` API. It filters for `DiagnosticSeverity.Error`, extracts the error message and line number, and passes this context to the `gpt-4o` model to generate a simplified, non-technical explanation.

3.8 Challenges Overcome

Handling API latency or VS Code API limitations.

3.9 Evaluation and Methodology

3.9.1 Experimental Design

The study employs a **within-subjects comparative design**. Participants will perform two standardised debugging and project-planning tasks:

1. **Control Condition:** Using a standard ("Vanilla") VS Code installation.
2. **Experimental Condition:** Using VS Code equipped with the Accessibility Plugin.

3.9.2 Qualitative Frameworks

To capture the subjective experience of neurodivergent users, two validated instruments are used:

- **NASA-TLX (Task Load Index):** This is critical for measuring the "Mental Demand" and "Frustration" sub-scales, which are often higher for developers with ADHD or Dyslexia.
- **System Usability Scale (SUS):** Provides a global view of the plugin's reliability and ease of integration into a professional workflow.

4 Results and Discussion

4.1 Analysis of NASA-TLX Scores

Did the plugin actually lower "Mental Demand"?

4.2 Task Initiation Findings

Comparison of start times for complex tasks.

4.3 Discussion

How AI-assisted decomposition mitigates hyperfocus/over-engineering.

4.4 Limitations

The subjectivity of self-testing and LLM hallucinations.

References

[1] Linus Bengtsson and Jimmy Hammarberg. *Web Content Accessibility Guidelines (WCAG) within a Company*. 2025.

[2] Jessica Bramham, Fiona Ambery, Susan Young, Robin Morris, Ailsa Russell, Kiriakos Xenitidis, Philip Asherson, and Declan Murphy. "Executive functioning differences between adults with attention deficit hyperactivity disorder and autistic spectrum disorder in initiation, planning and strategy formation". In: *Autism* 13.3 (2009), pp. 245–264.

[3] A. Cao, K. K. Chintamani, A. K. Pandya, and R. D. Ellis. *NASA TLX: Software for assessing subjective mental workload.* [Online; accessed 2026-02-25]. 2009. URL: https://psycnet.apa.org/record/2011-00741-013.

[4] Kiev Gama, Grischa Liebel, Miguel Goulão, Aline Lacerda, and Cristiana Lacerda. *A socio-technical grounded theory on the effect of cognitive dysfunctions in the performance of software developers with ADHD and autism*. IEEE, 2025.

[5] Philip G Griffiths, Robert H Taylor, Lisa M Henderson, and Brendan T Barrett. "The effect of coloured overlays and lenses on reading: a systematic review of the literature". In: *Ophthalmic and Physiological Optics* 36.5 (2016), pp. 519–544.

[6] Connor Harwood. *Navigating the Brain-AI Loop | Remtek Workplace*. [Online; accessed 2026-03-16]. Mar. 2026. URL: https://remtekworkplace.com/knowledge-hub/the-digital-ramp-and-the-cognitive-debt-navigating-the-brain-ai-loop/.

[7] M. Hyzy, R. Bond, M. Mulvenna, L. Bai, A. Dix, and Hunt Leigh S. *JMIR mHealth and uHealth - System Usability Scale Benchmarking for Digital Health Apps: Meta-analysis*. [Online; accessed 2026-02-25]. Aug. 2022. URL: https://mhealth.jmir.org/2022/8/e37290.

[8] Grischa Liebel, Noah Langlois, and Kiev Gama. "Challenges, strengths, and strategies of software engineers with ADHD: A case study". In: *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society*. 2024, pp. 57–68.

[9] Meredith Ringel Morris, Andrew Begel, and Ben Wiedermann. *Understanding the challenges faced by neurodiverse software engineering employees: Towards a more inclusive and productive technical workforce*. 2015.

[10] Kaia Newman, Sarah Snay, Madeline Endres, Manasvi Parikh, and Andrew Begel. ""get me in the groove": A mixed methods study on supporting ADHD professional programmers". In: *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE. 2025, pp. 1217–1229.

[11] Meredith Ringel Morris. “AI and Accessibility: A Discussion of Ethical Considerations”. In: *arXiv e-prints* (2019), arXiv–1908.

[12] Inês Lazana Barros da Rocha. *A Conceptual Model on Neurodivergent Issues in Software Engineering*. 2025.

[13] Barak Rosenshine. “Principles of instruction: Research-based strategies that all teachers should know.” In: *American educator* 36.1 (2012), p. 12.

[14] Judy Singer. “Why can’t you be normal for once in your life? From a ’problem with no name’ to the emergence of a new category of difference”. In: *Disability Discourse*. Ed. by Mairian Corker and Sally French. Buckingham: Open University Press, 1999, pp. 59–67.

[15] John Sweller. *Cognitive load during problem solving: Effects on learning*. 1988.

[16] Pragya Verma, Marcos Vinicius Cruz, and Grischa Liebel. *Differences between neurodivergent and neurotypical software engineers: Analyzing the 2022 stack overflow survey*. Springer, 2025.